

STRING

1. Write a program to take a string as input and print it.
2. Find the length of a given string without using `length()`.
3. Convert a given string to uppercase and lowercase.
4. Concatenate two strings without using the `+` operator.
5. Check if a given string is empty or not.
6. Compare two strings using `'=='` and `'.equals()'` and explain the difference.
7. Check if a string contains a given substring.
8. Find the first and last occurrence of a character in a string.
9. Find the number of occurrences of a character in a string.
10. Reverse a string without using built-in methods.
11. Remove all vowels from a given string.
12. Replace all spaces in a string with `-`.
13. Remove duplicate characters from a string.
14. Count the number of words in a string.
15. Convert a string to a character array and print each character.
16. Convert a string representation of a number to an integer (`"123" → 123`).
17. Convert a character array back into a string.
18. Split a string based on spaces or a given delimiter.
19. Convert the first letter of each word in a string to uppercase.
20. Check if a string is a palindrome.
21. Find the longest word in a given sentence.
22. Extract digits from a given string (`"a1b2c3" → 123`).
23. Find and replace a substring in a string.
24. Sort the characters of a string in alphabetical order.
25. Reverse the words in a string (`"hello world" → "world hello"`).
26. Find the largest and smallest words in a given string.
27. Check if a given string consists of only digits.
28. Find the shortest word in any sentence.

29. Check if two strings are anagrams (for example, "listen" and "silent").
30. Find the longest substring without repeating characters.
31. Implement a custom method to trim leading and trailing spaces (without using `trim()`).
32. Compress a string using counts of repeated characters (example: "aabcccccaaa" → "a2b1c5a3").
33. Check if a string has all unique characters (kono method use na kore).
34. Implement your own version of the `indexOf()` method.
35. Capitalise the first and last letter of each word in a sentence.
36. Check if two strings are rotations of each other
For example: "ABCD" and "CDAB" → tahole true asbe.
37. Count and display the frequency of each character in a string.
38. Implement a method to perform basic string compression (remove consecutive duplicates and keep one occurrence only).
39. Remove a specified word from a sentence.
40. Identify and remove all special characters from a string.
41. Split a camelCase string into individual words
Example: "thisIsCamelCase" → "this Is Camel Case".
42. Count the number of digits, letters, and special characters in a string.
43. Convert a Roman numeral string to an integer.
44. Group all vowels at the beginning of a string and consonants at the end
Preserve relative order.
45. Check if a string is a valid palindrome, ignoring spaces and punctuation.
46. Mask all characters of a string except the last four.
Example: "1234567890" → "*****7890".
47. Remove all duplicate words from a sentence.
48. Count the number of valid email addresses from a sentence.
49. Replace every nth character in a string with a given character.
50. Find the first non-repeating character in a string.
51. Validate whether a string contains a valid IP address format.
52. Implement a function to repeat a string n times without using built-in methods.
53. Check if a string ends with a given substring without using `endsWith()`.
54. Count the number of palindromic words in a sentence.
55. Reverse characters in each word of a sentence without changing word order.
56. Identify and count the most frequently occurring word in a paragraph.

Prepared and Organized by
BADHON NANDI
Student Tutor & FYAT Mentor
Department of Computer Science and Engineering, Brac University

Editing in progress

Email me for any question additions/enquiries: badhon.nandi@g.bracu.ac.bd